

Computer Vision with Python — Chapter 2

Learning Objectives

- Understand point operations and intensity transformations.
- Interpret an image histogram and relate it to brightness and contrast.
- Apply contrast stretching and gamma correction.
- Understand and implement global histogram equalization.
- Understand why CLAHE is useful for local contrast enhancement.
- Compare enhancement methods quantitatively and visually using Python/OpenCV.

2.1 Why Image Enhancement?

Image enhancement modifies pixel intensities so that important structures become easier to see or process. Unlike image restoration, enhancement does not necessarily attempt to recover a physically exact original image. Its purpose is often to improve visibility, contrast, or suitability for a later computer-vision task.

In simple words, enhancement changes how the available image information is presented. For example, a dark image may contain useful information, but its intensity values may be concentrated in a narrow part of the available 0–255 range. By remapping those values, details can become easier to distinguish.

Enhancement is therefore task-dependent. A transformation that makes an image visually pleasing is not automatically the best transformation for measurement. In computer vision, we should ask whether the enhancement makes the relevant structures easier for a later algorithm to detect, segment, match, or classify.

Throughout this chapter, the input is treated as an 8-bit grayscale image. The same ideas can be extended to color images, but color enhancement requires extra care because independently changing channels can alter color relationships.

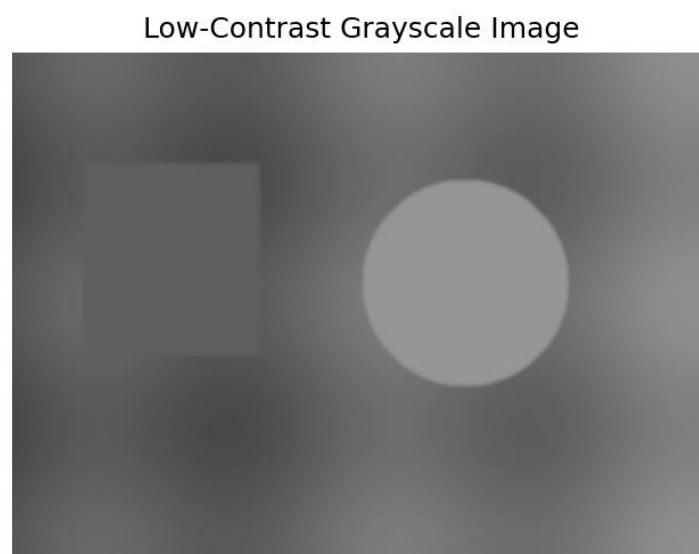


Figure 2.1 — Example low-contrast grayscale image used to illustrate expected outputs.

2.2 Point Operations

A point operation computes each output pixel using only the corresponding input pixel. If r is the input intensity and s is the output intensity, a general point transformation is written as:

$$s = T(r)$$

This means that the output value at a location does not directly depend on neighboring pixels. The transformation T tells us how an input intensity is mapped to a new output intensity.

Examples include brightness adjustment, image negative, thresholding, contrast stretching, and gamma correction.

Point operations are computationally simple because the same rule can often be applied independently to every pixel. NumPy is particularly useful here because one expression can operate on an entire image array without writing an explicit Python loop.

2.3 Image Negative

For an 8-bit grayscale image with intensity range 0–255:

$$s = 255 - r$$

The transformation reverses brightness. A pixel of value 0 becomes 255, 50 becomes 205, and 255 becomes 0. This is useful for understanding intensity transformations and can also make certain structures more visible when light and dark relationships are reversed.

```
import numpy as np
import matplotlib.pyplot as plt
import cv2
from google.colab import files

# Upload the image
uploaded = files.upload()

# Read the image in grayscale
img = cv2.imread("chapter2_flower.jpg", cv2.IMREAD_GRAYSCALE)

# Check whether the image was loaded
if img is None:
    print("Image could not be loaded.")
else:
    # Create the negative image
    negative = 255 - img

    # Display original and negative images
    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.imshow(img, cmap="gray", vmin=0, vmax=255)
    plt.title("Original Image")
    plt.axis("off")

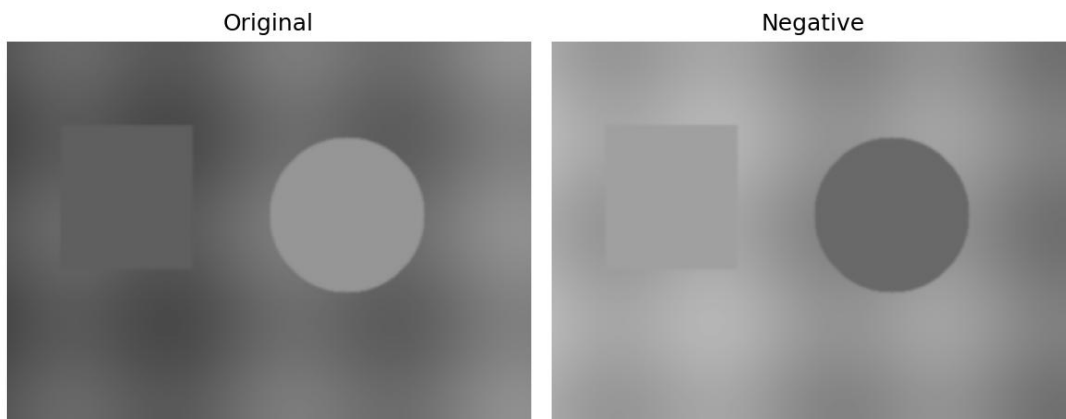
    plt.subplot(1, 2, 2)
    plt.imshow(negative, cmap="gray", vmin=0, vmax=255)
    plt.title("Negative Image")
    plt.axis("off")
```

```
plt.show()
```

Line-by-line / step-by-step explanation

1. ``import cv2`` loads OpenCV.
2. ``cv2.imread('chapter2_flower.jpg', cv2.IMREAD_GRAYSCALE)`` reads the uploaded image directly as a single-channel grayscale image.
3. ``negative = 255 - img`` applies the negative transformation to every pixel. NumPy/OpenCV image arrays support this element-wise operation.
4. ``plt.figure(figsize=(10, 4))`` creates a figure large enough to display the original and negative images side by side in Google Colab.
5. ``plt.subplot(1, 2, 1)`` and ``plt.subplot(1, 2, 2)`` create two display positions: one for the original image and one for the negative image.
6. ``plt.imshow(..., cmap='gray', vmin=0, vmax=255)`` displays each image using a fixed grayscale intensity range, making the comparison consistent.
7. ``plt.show()`` displays the completed comparison inside the Colab notebook. Unlike ``cv2.imshow()``, it does not require a separate desktop window.

Expected output



Expected output: bright regions become dark and dark regions become bright.

Observation: bright regions become dark and dark regions become bright.

2.4 Histograms

A grayscale histogram counts how many pixels occur at each intensity level. A histogram concentrated toward the left usually indicates a darker image; one concentrated toward the right indicates a brighter image. A narrow histogram often indicates low contrast, while a distribution spread across a wider intensity range often indicates higher contrast.

The horizontal axis represents intensity, usually from 0 to 255 for an 8-bit image. The vertical axis represents frequency: how many pixels have a particular intensity or fall within a small intensity interval.

A histogram tells us about the distribution of intensities, but it does not tell us where those intensities occur. Two images with very different spatial structures can therefore have similar or even identical histograms.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("chapter2_flower.jpg", cv2.IMREAD_GRAYSCALE)

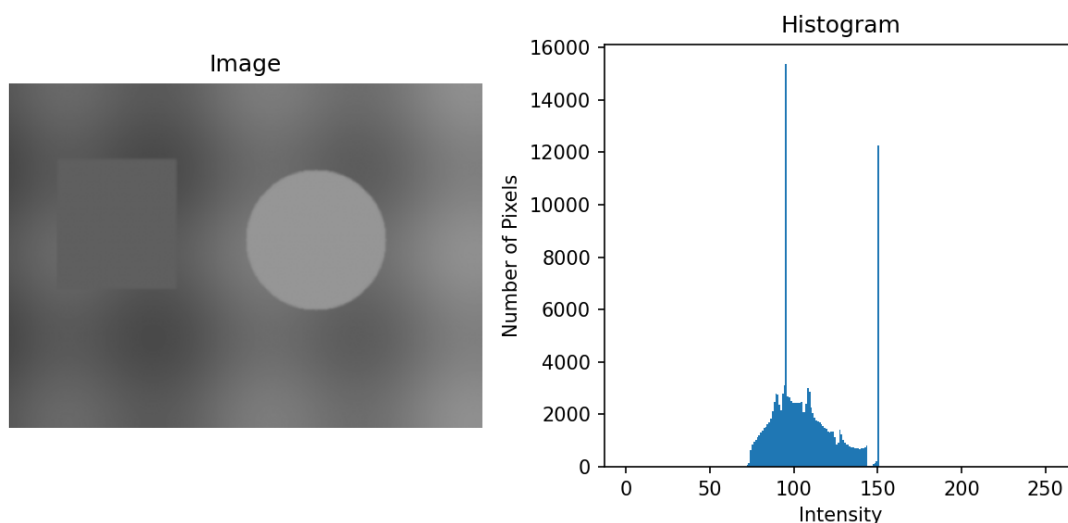
plt.imshow(img, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()

plt.hist(img.ravel(), bins=256, range=[0, 256])
plt.title("Image Histogram")
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.show()
```

Line-by-line / step-by-step explanation

1. `matplotlib.pyplot` is imported as `plt` for image and graph visualization.
2. The image is read as grayscale, so each pixel contributes one intensity value.
3. `plt.imshow(..., cmap='gray')` displays the two-dimensional array using a grayscale colormap.
4. `plt.axis('off')` hides plot axes for the image display.
5. `img.ravel()` converts the 2-D image array into a one-dimensional sequence of pixel values.
6. `bins=256` requests one histogram bin for each possible 8-bit intensity level.
7. `range=[0,256]` fixes the histogram intensity interval.
8. The x-axis is labeled intensity and the y-axis is the number of pixels.

Expected output



Expected output: the image and its intensity histogram. The example histogram occupies a relatively restricted part of the 0–255 range.

2.5 Contrast Stretching

Contrast stretching expands a restricted intensity range into a wider range. For min–max normalization to 0–255:

$$s = ((r - r_{\min}) / (r_{\max} - r_{\min})) \times 255$$

Here $r_{\min}$ is the minimum intensity present in the input image and $r_{\max}$ is the maximum. The minimum is mapped to 0, the maximum is mapped to 255, and intermediate values are linearly distributed between them.

The method is useful when the image uses only a limited portion of the available dynamic range. However, if a few extreme outliers determine $r_{\min}$ or $r_{\max}$, ordinary min–max stretching may not produce the most useful result. Percentile-based stretching is one possible extension.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("chapter2_flower.jpg", cv2.IMREAD_GRAYSCALE)

r_min = img.min()
r_max = img.max()

stretched = (
    (img.astype(np.float32) - r_min)
    * 255.0 / (r_max - r_min)
)

stretched = np.clip(stretched, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(stretched, cmap="gray", vmin=0, vmax=255)
plt.title("Contrast Stretched")
plt.axis("off")
plt.show()
```

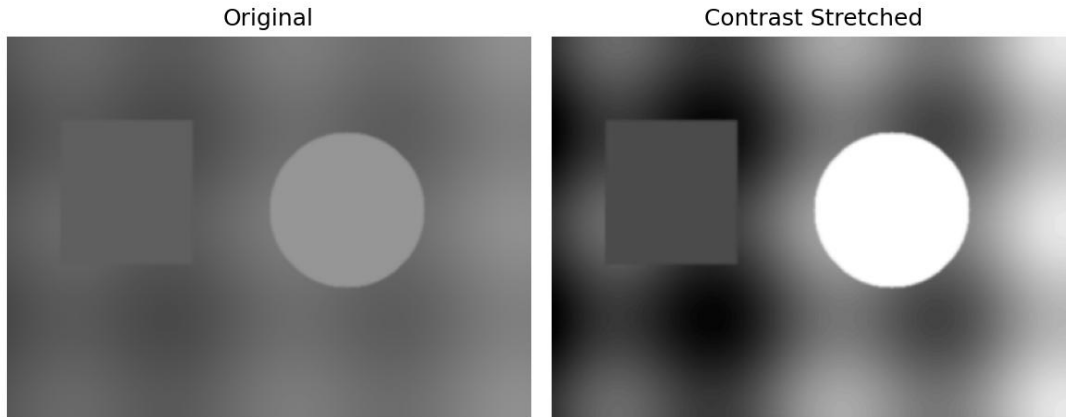
Line-by-line / step-by-step explanation

1. `img.min()` finds the smallest intensity present and `img.max()` finds the largest.
2. `img.astype(np.float32)` converts the unsigned 8-bit array to floating point before subtraction and division.
3. Subtracting `r_min` shifts the observed minimum intensity to zero.
4. Dividing by `(r_max-r_min)` normalizes the observed range.
5. Multiplying by 255 expands the normalized values to the standard 8-bit display range.
6. `np.clip` prevents numerical values outside 0–255.
7. `astype(np.uint8)` converts the processed floating-point array back to an 8-bit image.

8. The two OpenCV windows allow direct visual comparison.

Expected output

Example input minimum = 72
Example input maximum = 150
Stretched minimum = 0
Stretched maximum = 255



Expected output: the restricted input range is expanded, producing stronger global contrast.

Important: convert to floating point before subtraction/division to avoid unintended unsigned-integer arithmetic.

2.6 Gamma Correction

Gamma correction uses a nonlinear power-law transformation:

$$s = c \cdot r^\gamma$$

For normalized intensities between 0 and 1, $\gamma < 1$ generally brightens mid-tones, while $\gamma > 1$ generally darkens them.

Unlike linear contrast stretching, gamma correction does not expand all parts of the intensity range equally. It changes the shape of the intensity mapping. This makes gamma useful when we specifically want to control darker or brighter tonal regions.

When $\gamma = 1$, the transformation is the identity transformation: the output is the same as the input, apart from possible rounding.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("chapter2_flower.jpg", cv2.IMREAD_GRAYSCALE)

gamma = 0.5
normalized = img.astype(np.float32) / 255.0
corrected = np.power(normalized, gamma)
corrected = np.clip(corrected * 255, 0, 255).astype(np.uint8)

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
```

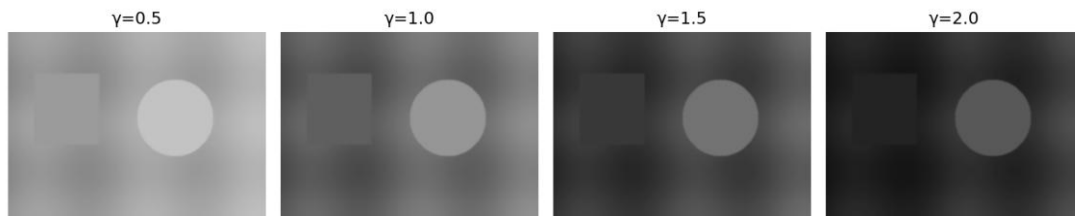
```
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(corrected, cmap="gray", vmin=0, vmax=255)
plt.title("Gamma Corrected")
plt.axis("off")
plt.show()
```

Line-by-line / step-by-step explanation

1. The image is read as an 8-bit grayscale array.
2. `gamma = 0.5`` selects a gamma value below one.
3. `astype(np.float32) / 255.0`` converts intensities to floating point and normalizes them to approximately $[0,1]$.
4. `np.power(normalized, gamma)`` raises every normalized pixel to the selected power.
5. Because gamma is 0.5, many mid-range values increase, so the image becomes brighter.
6. The result is multiplied by 255 to return to the conventional 8-bit scale.
7. `np.clip`` protects the valid range and `astype(np.uint8)`` converts back to an image suitable for display/storage.

Expected output



Expected output for $\gamma = 0.5, 1.0, 1.5$ and 2.0 . Values below 1 brighten mid-tones; values above 1 darken them.

2.7 Histogram Equalization

Histogram equalization uses the cumulative distribution of image intensities to remap pixels and improve global contrast. It is especially useful when useful information occupies a limited intensity range.

The basic idea is to calculate the histogram, convert it into a cumulative distribution function (CDF), and use that CDF as an intensity mapping. Frequently occurring ranges are redistributed so that the output uses the available intensity range more broadly.

The output histogram does not have to become perfectly flat. Digital images contain a finite number of pixels and discrete intensity levels, so exact uniformity is generally not expected.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread(
    "chapter2_flower.jpg",
    cv2.IMREAD_GRAYSCALE
```

```

)

equalized = cv2.equalizeHist(img)

plt.figure()
plt.imshow(equalized, cmap="gray")
plt.title("Histogram Equalized Image")
plt.axis("off")
plt.show()

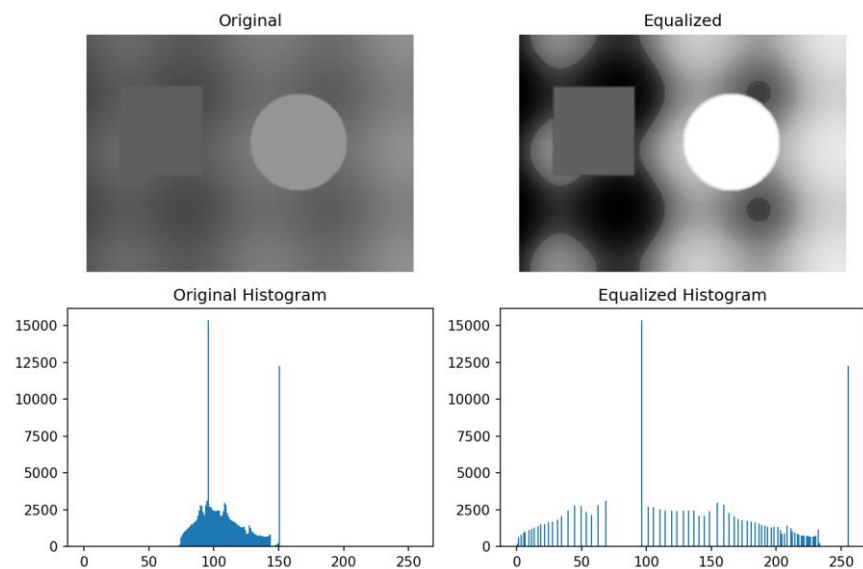
plt.figure()
plt.hist(
    equalized.ravel(),
    bins=256,
    range=[0, 256]
)
plt.title("Equalized Histogram")
plt.xlabel("Intensity")
plt.ylabel("Number of Pixels")
plt.show()

```

Line-by-line / step-by-step explanation

1. The grayscale image is loaded.
2. `cv2.equalizeHist(img)` performs global histogram equalization. The input should be a single-channel 8-bit image for this standard call.
3. A new Matplotlib figure displays the equalized image.
4. `equalized.ravel()` converts the processed image into a one-dimensional list of intensity values.
5. The second figure plots the histogram after equalization.
6. Comparing the original and equalized histograms helps explain why the visual contrast changed.

Expected output



Expected output: the equalized image and a histogram redistributed over a wider intensity range.

Limitation: global equalization can over-enhance noise or produce unnatural contrast because the same global mapping is used everywhere.

2.8 CLAHE

CLAHE stands for Contrast Limited Adaptive Histogram Equalization. Instead of applying one transformation to the entire image, it operates on small regions (tiles). Contrast limiting reduces excessive amplification, especially in relatively uniform regions.

Adaptive processing is useful when different parts of an image have different illumination or contrast. A single global mapping may improve one region while leaving another region poorly enhanced.

The `clipLimit` controls how strongly local histogram peaks are limited before redistribution. `tileGridSize` controls how the image is divided into local regions. These parameters should be selected based on the image and application rather than treated as universal constants.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("chapter2_flower.jpg", cv2.IMREAD_GRAYSCALE)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
clahe_img = clahe.apply(img)

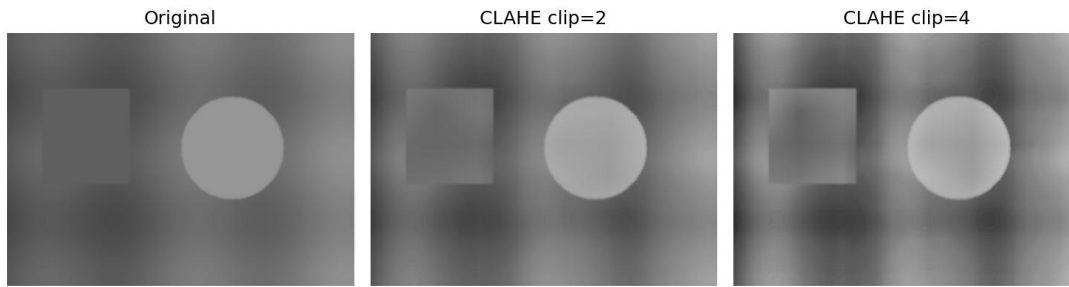
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.imshow(img, cmap="gray", vmin=0, vmax=255)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(clahe_img, cmap="gray", vmin=0, vmax=255)
plt.title("CLAHE")
plt.axis("off")
plt.show()
```

Line-by-line / step-by-step explanation

1. The input image is loaded in grayscale.
2. `cv2.createCLAHE(...)` creates a CLAHE processor.
3. `clipLimit=2.0` limits local contrast amplification.
4. `tileGridSize=(8,8)` divides the image into a grid of local tiles for processing.
5. `clahe.apply(img)` performs the adaptive equalization and interpolates between neighboring tile mappings to reduce visible boundaries.
6. The original and CLAHE images are displayed for comparison.

Expected output



Expected output: CLAHE improves local contrast. A larger clip limit can produce stronger enhancement but may also amplify noise or make the result harsher.

2.9 Comparing the Methods

The enhancement methods in this chapter solve related but different problems. Selecting a method should depend on the intensity distribution and on whether the desired change is global, nonlinear, or local.

Method	Type	Main Use	Key Limitation
Contrast stretching	Global / point	Expand dynamic range	Sensitive to extreme min/max values
Gamma correction	Point / nonlinear	Control brightness and mid-tones	Gamma must be chosen appropriately
Histogram equalization	Global statistical	Improve global contrast	May over-enhance noise
CLAHE	Local adaptive	Improve local contrast	Parameters affect appearance and computation

A useful workflow is to inspect the original image and histogram first. If the entire image occupies a narrow range, contrast stretching may be sufficient. If mid-tone brightness is the main issue, gamma correction is appropriate. If the global intensity distribution is poor, histogram equalization may help. If contrast varies strongly across the image, CLAHE is often a better candidate.

2.10 Complete Comparison Program

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread(
    "chapter2_flower.jpg",
    cv2.IMREAD_GRAYSCALE
)

# Contrast stretching
r_min, r_max = img.min(), img.max()
stretch = (
    (img.astype(np.float32) - r_min)
    * 255.0 / (r_max - r_min)
)
stretch = np.clip(
    stretch, 0, 255
).astype(np.uint8)
```

```

# Gamma correction
gamma = 0.5
gamma_img = np.power(
    img.astype(np.float32) / 255.0,
    gamma
)
gamma_img = np.clip(
    gamma_img * 255, 0, 255
).astype(np.uint8)

# Histogram equalization
hist_eq = cv2.equalizeHist(img)

# CLAHE
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)
clahe_img = clahe.apply(img)

images = [
    img, stretch, gamma_img,
    hist_eq, clahe_img
]

titles = [
    "Original", "Stretching", "Gamma",
    "Hist. Equalization", "CLAHE"
]

for image, title in zip(images, titles):
    plt.figure()
    plt.imshow(image, cmap="gray")
    plt.title(title)
    plt.axis("off")
    plt.show()

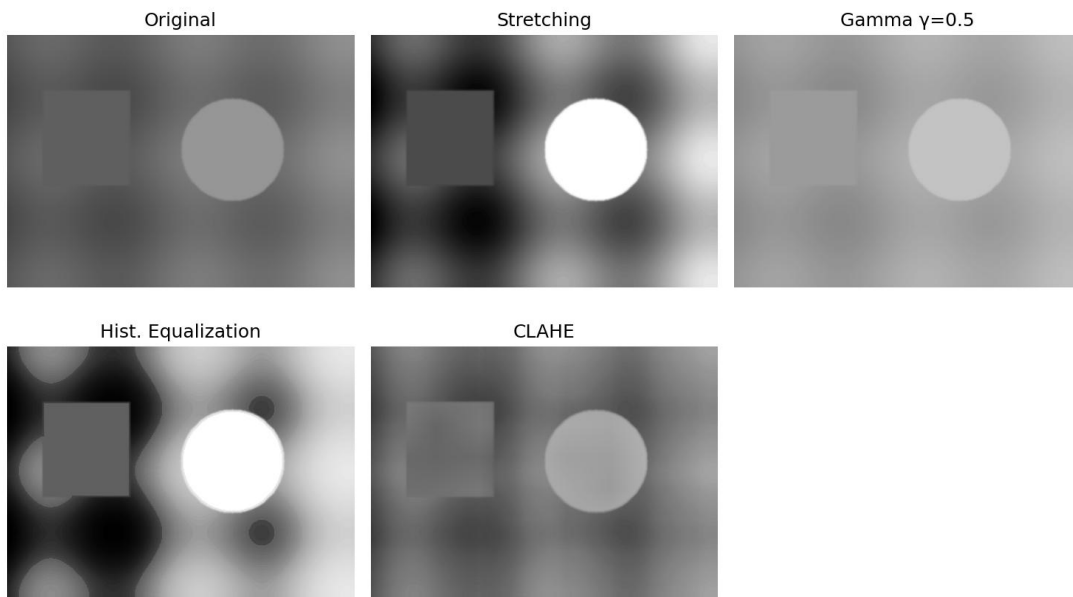
```

Line-by-line / step-by-step explanation

1. The required libraries are imported.
2. The input image is loaded once in grayscale so every enhancement method receives the same data.
3. The contrast-stretching block finds the observed minimum and maximum, converts to float, normalizes the range and converts back to uint8.
4. The gamma block normalizes the image to [0,1], applies the power-law transformation and converts back to [0,255].
5. `cv2.equalizeHist`` creates the globally equalized result.
6. `createCLAHE`` and `apply`` create the locally adaptive result.
7. The five images are placed in one Python list and their corresponding names in a second list.
8. `zip(images, titles)`` pairs each processed image with its title.
9. The loop creates a separate Matplotlib figure for every result.

10. Using the same grayscale display convention makes the visual comparison more meaningful.

Expected output



Expected comparison: each method changes the same low-contrast input differently. Stretching expands the range, gamma brightens mid-tones for $\gamma=0.5$, histogram equalization redistributes global intensities, and CLAHE emphasizes local contrast.

Optional quantitative comparison

Visual inspection is important, but it is also useful to inspect simple statistics. Mean intensity indicates average brightness, while standard deviation gives a basic measure of intensity spread. A larger standard deviation does not automatically mean a better image, but it can help describe how strongly the intensity distribution changed.

```
methods = {
    "Original": img,
    "Stretching": stretch,
    "Gamma": gamma_img,
    "HistEq": hist_eq,
    "CLAHE": clahe_img
}

for name, image in methods.items():
    print(
        name,
        "min =", image.min(),
        "max =", image.max(),
        "mean =", round(float(image.mean()), 2),
        "std =", round(float(image.std()), 2)
    )
```

Line-by-line / step-by-step explanation

1. A dictionary stores each method name together with its image.
2. `.items()` returns the name and image together during each loop iteration.
3. `min` and `max` report the observed range.`

4. ``mean`` reports average brightness.
5. ``std`` reports the spread of intensity values.
6. ``round(...,2)`` makes floating-point statistics easier to read.

Expected output: five lines of numerical statistics. The exact values depend on the selected image, so there is no single universal numerical answer.

2.11 Exercises

1. Load a grayscale image and display its minimum, maximum, mean, and standard deviation.
2. Plot the image histogram and explain whether the image is dark, bright, low contrast, or high contrast.
3. Create the negative of the image without using `cv2.bitwise_not()`.
4. Implement min–max contrast stretching manually using NumPy.
5. Apply gamma correction using $\gamma = 0.5, 1.0, 1.5$, and 2.0 . Explain the visual differences.
6. Apply global histogram equalization and compare the histogram before and after processing.
7. Apply CLAHE with at least two different `clipLimit` values and explain the difference.
8. Compare original, contrast-stretched, gamma-corrected, histogram-equalized, and CLAHE images.

2.12 Conceptual Questions

1. What is a point operation?
2. What information does an image histogram provide?
3. Why can two different images have similar histograms?
4. What happens when $\gamma < 1$ and $\gamma > 1$ in gamma correction?
5. Why can histogram equalization amplify noise?
6. How is CLAHE different from global histogram equalization?
7. Why should uint8 images often be converted to float before arithmetic operations?